Class 22: Requirements Engineering for ML Systems

Data-driven Systems Engineering

What should we build, and how will we know it works?

From need to behavior

Turn a stakeholder need into observable system behavior.

Kinds of requirement

Distinguish functional requirements from quality and domain constraints.

Acceptance checks

Define acceptance checks for software and ML behavior.

Key idea. A strong ML model does not rescue unclear goals. Requirements decide what "works" means before anyone trains anything.

RUNNING EXAMPLE

A hypothetical support service uses ML to suggest a category for incoming tickets. An operator reviews the suggestion before routing the ticket.

ticket → ML suggests → operator reviews → ticket routed

NEW **Stakeholder:** anyone who needs, uses, or is affected by the system. **Requirement:** a service the system shall provide, or a constraint on it, that can be checked. **Acceptance check:** evidence that a requirement is met.

Roadmap: requirements engineering is a process, not a document

ACTIVITY	WHAT IT DOES	THE ML TWIST
1. Elicitation	Find out what stakeholders need, and analyze and resolve conflicts between needs.	Labels, data access, and error costs are stakeholder questions too.
2. Specification	Write needs as precise statements that can be checked.	Quality is stated as an evaluation contract, not as a promise.
3. Validation	Check that the requirements describe the system stakeholders want.	The checks rely on data, so the data needs review too.
4. Management	Keep requirements consistent and traceable as things change.	New data, labels, and models change what "good" means.

Key idea. The activities overlap and repeat: validation and change send us back to elicitation. A requirements document is a snapshot of an ongoing agreement.

NEW **Elicitation:** discovering needs by asking, observing, and prototyping. **Requirements validation:** checking the requirements describe what stakeholders want. **Requirements management:** keeping requirements consistent.

1 Understand the need

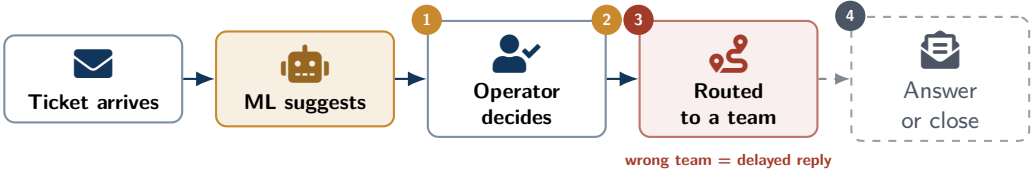


Previously: class 20 asked what a system must do and how well; now we make that precise.

IN THIS PART

- Start with the decision, not the technology
- From user need to system requirements
- Three types of requirement

Start with the decision, not the technology



1 Who uses the output?
A support operator.

2 What decision does it support?
Which team should receive a ticket.

3 What goes wrong?
A wrong team means a delayed response.

4 What is outside scope?
It does not answer or close tickets.

User need. Help operators route tickets consistently while retaining control over the final decision.

Key idea. Requirements attach to the decision and its consequences, not to the technology (here, an ML model). Scope says what we promise not to do.

NEW **Decision support:** a system that informs a human decision instead of making it. **Scope:** what the system will do, and what it will not.

From user need to system requirements



USER REQUIREMENT

Operators need help categorizing incoming support tickets.

refine into statements that can be checked

TRIGGER AND CONDITION	OBLIGATION	OBSERVABLE RESULT	CONSTRAINT
When an operator submits a ticket with a non-empty subject and body,	the system **shall**	display one suggested category	from the approved category list.

User requirements communicate the need, in the stakeholder's language and for the stakeholder.

System requirements specify behavior and constraints that can be checked. One statement, one check.

NEW **User requirement:** a need in the stakeholder's own words. **System requirement:** a checkable statement of behavior or constraint. **Shall / should:** shall marks a mandatory statement, should a desirable one.

Three types of requirement

	Functional what the system does	**Non-functional** how well it does it	**Domain** rules from the application field
Users in their words	*I want a suggested team for each ticket, and the last word.*	*Suggestions must not slow me down.*	*Tickets must only go to teams the organization recognizes.*
System in checkable terms	Suggest a category; allow the operator to accept or override it.	Return a suggestion within a stated latency under a stated load.	Use only categories in the support organization's approved taxonomy.

Two different questions. User/system is the audience and the level of detail. Functional/non-functional/domain is the kind of expectation.

Domain requirements are easy to miss: stakeholders take them for granted, and a system that ignores them is unusable however well it performs.

NEW **Domain requirement:** a rule that comes from the field the system works in, not from a user's wish. **Quality attribute:** a measurable property such as speed or reliability (Class 20). **Taxonomy:** the agreed list of categories.

2 Make requirements testable



Previously: a need becomes a system requirement, and requirements come in three kinds.

IN THIS PART

- Replace vague wishes with testable statements
- A user story needs acceptance criteria
- ML quality needs an evaluation contract
- Specify uncertainty and failure behavior

Replace vague wishes with testable statements

Weak

The classifier should be **fast** and **accurate**.

How fast, for how many users? Accurate on which tickets, measured how?

R1: latency requirement

With 20 concurrent users and ticket bodies up to 5,000 characters, at least 95% of suggestion requests shall complete within 2 seconds, measured over a 30-minute load test.

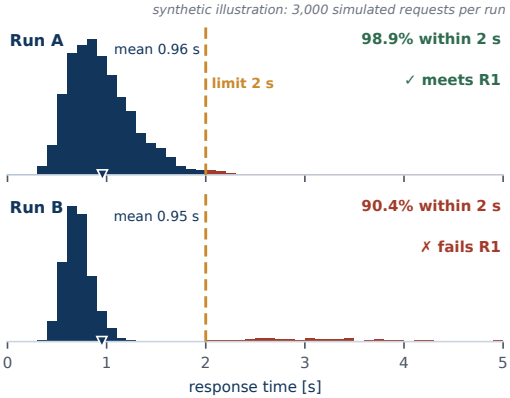
CONDITION	OUTCOME	THRESHOLD	EVIDENCE
20 users, ticket bodies up to 5,000 characters	a suggestion completes within 2 s	at least 95% of requests	a 30-minute load test

Testable means two independent people would run the check and agree on pass or fail.

The numbers are hypothetical targets to negotiate with stakeholders, not universal defaults.

NEW **Testable:** an observable result can show that the statement passes or fails. **Threshold:** the value that separates pass from fail.

What R1 checks: the share of requests within 2 seconds



Both runs average about **one second**.

R1 asks for 95% within 2 s: run A **passes**, run B **fails** because of its tail.

Name the statistic in the requirement: a share or a percentile, not an average.

Percentile. The 95th percentile (p95) is the value below which 95% of requests fall. "p95 $\leq$ 2 s" and "95% within 2 s" say the same thing.

NEW **Tail:** the slowest few requests, which an average hides. **Percentile:** the value below which a given share of observations falls.

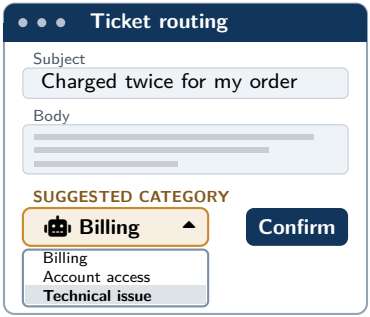
A user story needs acceptance criteria

USER STORY

As a **support operator**, I want to **correct a suggested category** so that **the ticket reaches the right team**.

ACCEPTANCE CRITERION (GIVEN, WHEN, THEN)

GIVEN	a ticket with a suggested category
WHEN	the operator selects another approved category and confirms
THEN	the system saves the selected category and records that an override occurred



Mock-up: the operator overrides the suggestion.

Key idea. A story gives the why; the criteria give the proof. Cover failure and boundary cases, and describe the result without prescribing the implementation.

NEW **User story:** one sentence stating a need as role, goal, and benefit. **Given–When–Then:** acceptance criteria as starting state, action, expected result. **Boundary case:** an input at the edge of what is allowed.

ML quality needs an evaluation contract

R2 *Before release, macro-F1 shall be at least 0.80 on a held-out set of 1,000 manually labeled tickets sampled from the previous month, with no tickets from the same conversation in training.*

AN EVALUATION CONTRACT NAMES FIVE THINGS

Labels	Population	Metric	Threshold	Procedure
manually labeled tickets	1,000 tickets from the previous month	macro-F1	at least 0.80	held out; conversations kept apart

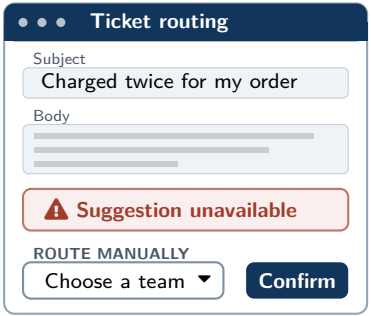
Macro-F1. Per-category F1 (a balance of precision and recall), averaged with equal weight: rare categories count as much as common ones.

Leakage. Tickets from one conversation are near-duplicates. Split across training and test, they inflate the score.

Offline is not online. A pass on last month's tickets does not guarantee future performance: the data may drift.

NEW **Macro-F1:** F1 computed for each category, then averaged with equal weight. **Data leakage:** information from the test data reaching training, which inflates the score.

Specify uncertainty and failure behavior



Mock-up: the scoring service is down.

ML model. The trained classifier that suggests a category for a ticket. The **scoring service** is the program that runs it.

Three questions to answer in the requirements

- What happens when the input is missing or too long?
- What happens when the ML model is unavailable?
- When the model is unsure, can the system defer to a human instead of guessing a category?

R3 fallback. *If the scoring service is unavailable, the interface shall display "Suggestion unavailable" and allow the operator to route the ticket manually.*

Acceptance check. Simulate a scoring-service outage and verify that manual routing still works.

Key idea. A wrong ML answer looks like a right one: say what happens when the model is absent, unsure, or wrong.

NEW **ML model:** the trained classifier that maps a ticket to a category. **Fallback:** behavior when the preferred path is unavailable. **Abstention:** declining to answer when unsure.

3 Validate and manage requirements



Previously: a requirement states a condition, an outcome, a threshold, and the evidence.

IN THIS PART

- Discover missing requirements before implementation
- Make conflicts and priorities explicit
- An example requirement form (IEEE style), and what changes
- Review requirements with five questions

Discover missing requirements before implementation

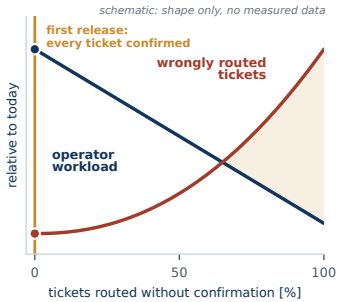
TECHNIQUE	WHAT WE DO OR ASK	WHAT IT EXPOSES
Interview and observe	Interview operators and watch how tickets are routed today.	The real practice, and its workarounds
Cost of errors	Ask which routing errors are most costly and which categories are ambiguous.	Priorities and hard cases
Label ownership	Check who defines the labels and resolves disagreements.	Who owns the ground truth
Data and privacy	Ask which ticket data may be displayed, stored, and accessed.	Privacy and access rules
Prototype walk-through	Walk through a simple prototype with operators.	Hidden assumptions

Key idea. Use several techniques: stakeholders cannot always say what they know (tacit knowledge), so observation and prototypes reveal what interviews miss.

NEW **Tacit knowledge:** what experts know and do but cannot easily put into words. **Prototype:** a rough, quick version used to get feedback early. **Ambiguous category:** a ticket that could belong to more than one category.

Make conflicts and priorities explicit

⚖ Example conflict. Automatically routing more tickets may reduce manual work while increasing incorrect routing.



1. **AGREE:** which errors matter most, and who accepts the trade-off.
2. **DECIDE:** for example, keep operator confirmation for every ticket in the first release.
3. **RECORD:** the decision, its owner, and the evidence needed to revisit it.

PRIORITIZE

Essential: manual fallback → **Optional:** convenience features

NEW **Stakeholder conflict:** two needs or goals that cannot both be fully satisfied. **Decision record:** a short note of what was decided, by whom, and why. **Priority:** the order in which requirements matter when time is short.

Example: a requirement form in the IEEE style

Requirement record EXAMPLE: HYPOTHETICAL VALUES

ID AND TITLE	**R1** Suggestion latency (non-functional)
STATEMENT	With 20 concurrent users and ticket bodies up to 5,000 characters, at least 95% of suggestion requests **shall** complete within 2 seconds.
RATIONALE	Operators route tickets while customers wait; slow suggestions would be ignored.
SOURCE	Support operators (interviews)
PRIORITY	High, to be negotiated | Status: proposed
VERIFICATION	**Test:** a 30-minute load test; passes if at least 95% of requests finish within 2 s.

Where it comes from

ISO/IEC/IEEE 29148 (2018) replaced IEEE 830-1998. Its SRS template groups requirements by kind and ends with verification.

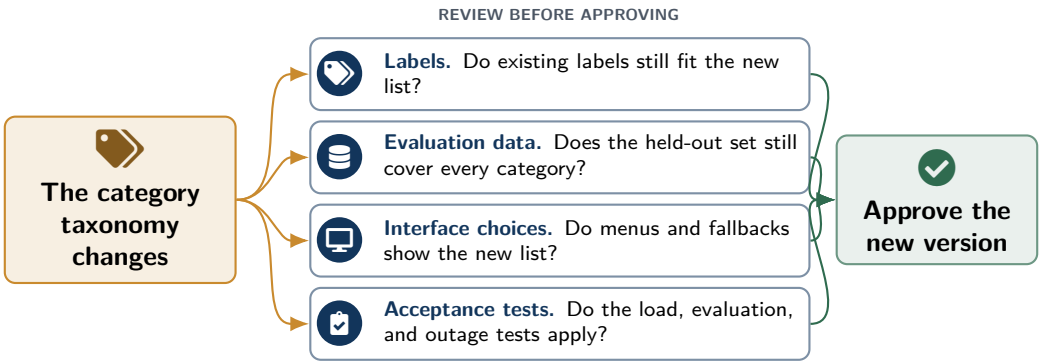
Four verification methods

Test: run it and measure (R1).
Demonstration: operate, observe.
Inspection: examine an artifact.
Analysis: calculate or reason.

Key idea. A form makes every requirement carry its rationale, source, priority, and verification method: the data needed for traceability, negotiation, and change control.

NEW **SRS:** Software Requirements Specification, the document that lists a system's requirements. **Verification method:** how a requirement is checked: test, demonstration, inspection, or analysis. **Traceability:** links from a need to a requirement to its check.

When something changes: use the traces to find the impact



Key idea. Impact analysis is only possible if the links exist: each requirement connects to a need, to the data it depends on, and to a check.

NEW **Impact analysis:** finding everything a proposed change touches before approving it. **Volatile requirement:** one likely to change, such as a category list.

Review requirements with five questions

Necessary	Whose need does this support?
Unambiguous	Could two readers interpret it differently?
Testable	What evidence would establish that it passes?
Feasible	Do the data, time, and resources support it?
Traceable	Can we connect it to a need, a test, and future changes?

Validation of requirements asks: are these the right requirements? **Verification** asks: does the built system meet them? This review is validation; the acceptance checks are verification.

NEW **Feasible:** achievable with the available data, time, and resources. **Verification:** checking that the system meets its requirements.

Three ideas to keep



Need to check

A requirement connects a stakeholder need to a result that can be checked.



More than a model score

ML requirements cover evaluation conditions and human fallback as well as model scores.



Write it down

Write down priorities, acceptance evidence, and the impact of changes.

Further reading. Sommerville, *Software Engineering*: requirements engineering. User stories and acceptance criteria in the Atlassian guide.

AI-assisted programming: a good prompt is a good requirement

GOOD REQUIREMENT	BECOMES, IN THE PROMPT	WHY IT HELPS
Necessary	Need and scope: who uses it, what it must do, and what not to build or change.	**Stops over-building and unwanted edits.**
Unambiguous	One `shall` statement per line, with an ID, a threshold, and the behavior on failure.	**Checkable lines and fewer hidden guesses.**
Testable	"Write the tests first (Given–When–Then), run them, and report failures."	**"Looks right" becomes pass or fail.**
Feasible	Limits and resources: language, libraries, data, and performance targets.	**Fewer invented APIs and impossible targets.**
Traceable	"Cite the requirement ID in each change; list any requirement not covered."	**The diff can be reviewed against the spec.**

Key idea. Review the prompt with the same five questions, and keep the spec in the repository. The assistant fills every gap with a guess; checking stays your job.

NEW **Prompt:** the instructions and context given to a language model. **AI-assisted programming:** coding with an assistant that generates code. **Hidden assumption:** a gap the model fills with a guess.

From this class to the next

1 SE and ML | 2 Requirements | 3 Process | 4 Design | 5 Quality | 6 Evolution

THIS CLASS — NEXT: CLASS 23

What stays with us

- A requirement links a need to a result that can be checked.
- ML adds evaluation conditions and a fallback.
- A good prompt is a good requirement.

Use it in your project

- Give each requirement an ID, a threshold, and a check.
- Record rationale, source, and priority.
- Say what happens when the model fails.

Next: Class 23

Process Models and Agile Foundations.

How teams organize work across exploration, validation, and release, especially when requirements may change.

Course thread. Knowing what "good" means before we build now leads to how we organize the work (Class 23) and, later, how we run it in production (MLOps).

References

Textbook	Sommerville, I. *Software Engineering*, Pearson: the requirements engineering chapters.
Standards	ISO/IEC/IEEE 29148:2018. Systems and software engineering: Life cycle processes: Requirements engineering. It replaced IEEE Std 830-1998, the earlier SRS recommended practice.
Requirements for ML	Vogelsang, A. and Borg, M. (2019). Requirements Engineering for Machine Learning: Perspectives from Data Scientists. IEEE RE Workshops. Nahar, N. et al. Collaboration Challenges in Building ML-Enabled Systems (arXiv:2110.10234).
Stories and criteria	Atlassian. User stories: examples and a template. North, D. (2006). Introducing BDD: behavior-driven development and the Given–When–Then format.